MASTER ENGINEERING CASE STUDY PORTFOLIO
INDEX AND EXECUTIVE SYNTHESIS

Navigation Guide for 36 Technical and Leadership Case Studies

Author	Venkata Lakshmi Parimala Pasupuleti
Experience base	Approximately eight years in Affordable Housing production systems
Portfolio size	36 technical engineering and leadership case studies
Primary themes	Data, backend, architecture, reliability, delivery, AI, leadership, customer impact
Purpose	Provide one navigable map across the complete body of work
Evidence model	Direct experience separated from proposed modernization
Positioning	Senior Software Engineer - Data and AI

This index summarizes and connects the case studies. It does not replace the detailed documents and does not expand claims beyond the experience boundaries established in them.

Document Map

1. Executive Portfolio Summary
2. Engineering Identity
3. Portfolio Coverage Map
4. Complete Case Study Index
5. Data and Domain Engineering Collection
6. Backend, Architecture, and Scale Collection
7. Reliability and Operations Collection
8. Delivery, Migration, and Modernization Collection
9. AI Engineering Collection
10. Leadership and Business Impact Collection
11. Foundational Production Examples
12. Experience-to-Competency Matrix
13. Suggested Navigation Paths
14. Evidence and Claim Boundaries
15. Cross-Study Themes
16. Portfolio Strengths
17. Remaining Optional Extensions
18. Portfolio Maintenance Guide
19. Closing Synthesis

1. Executive Portfolio Summary

This portfolio presents a senior engineering profile built through long-term production ownership rather than isolated prototypes. The foundation is Affordable Housing: annual HUD income limits, eligibility, rent, HAP, tax-credit behavior, documents, migrations, support incidents, and cross-team dependencies. The collection then extends that foundation into scalable backend systems, asynchronous processing, cloud integration, technical leadership, and production AI architecture.

The 36 case studies are intentionally overlapping at the system boundaries but distinct in engineering focus. One document may explain the same production event as a database migration, another as a release-control problem, and another as an evidence-driven incident investigation. This is useful because senior engineering work is multidimensional: one change can affect data, architecture, delivery, reliability, customers, and team operations at the same time.

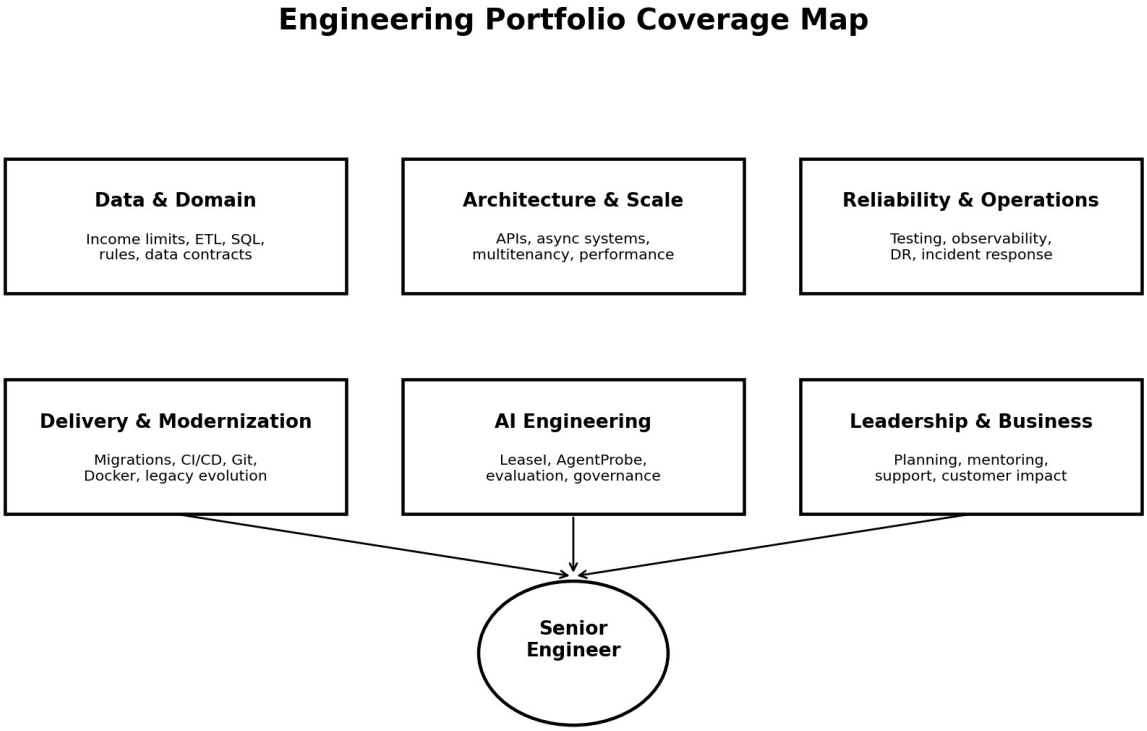


Figure 1. The portfolio combines six major capability groups around one senior-engineering identity.

2. Engineering Identity

The portfolio supports a consistent engineering identity: a backend and data-focused senior engineer who understands business-critical production systems, writes and diagnoses complex SQL, owns data and document workflows, works across teams, modernizes carefully, and applies AI only where evidence and safety controls are strong.

Identity dimension	Evidence across the portfolio
Data depth	Annual HUD income limits, large transformations, stored procedures, effective dating, validation, lineage, schema evolution

Identity dimension	Evidence across the portfolio
Backend systems	APIs, service integration, RabbitMQ, workers, retries, idempotency, document generation, object storage
Production ownership	Seven-year module depth, releases, incidents, migrations, support, validation, rollback
Architecture judgment	System design, trade-offs, multitenancy, scale, observability, security, reliability, cost
Modernization	Git adoption, Docker learning, AWS integration, legacy refactoring, CI/CD, technical debt
AI engineering	LeaseI investigation platform, AgentProbe evaluation, retrieval, tools, grounding, human escalation
Leadership	Planning, reviews, mentoring, knowledge transfer, customer impact, architecture governance

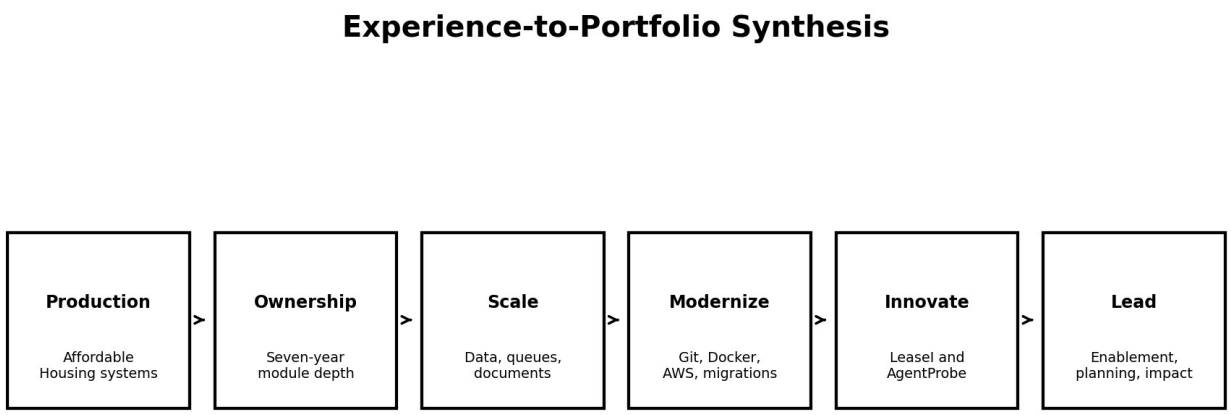


Figure 2. The case studies convert production depth into a complete senior-engineering narrative.

3. Portfolio Coverage Map

The portfolio is balanced across implementation depth, system design, operations, delivery, AI, and leadership. Some documents have one primary category and several secondary categories.

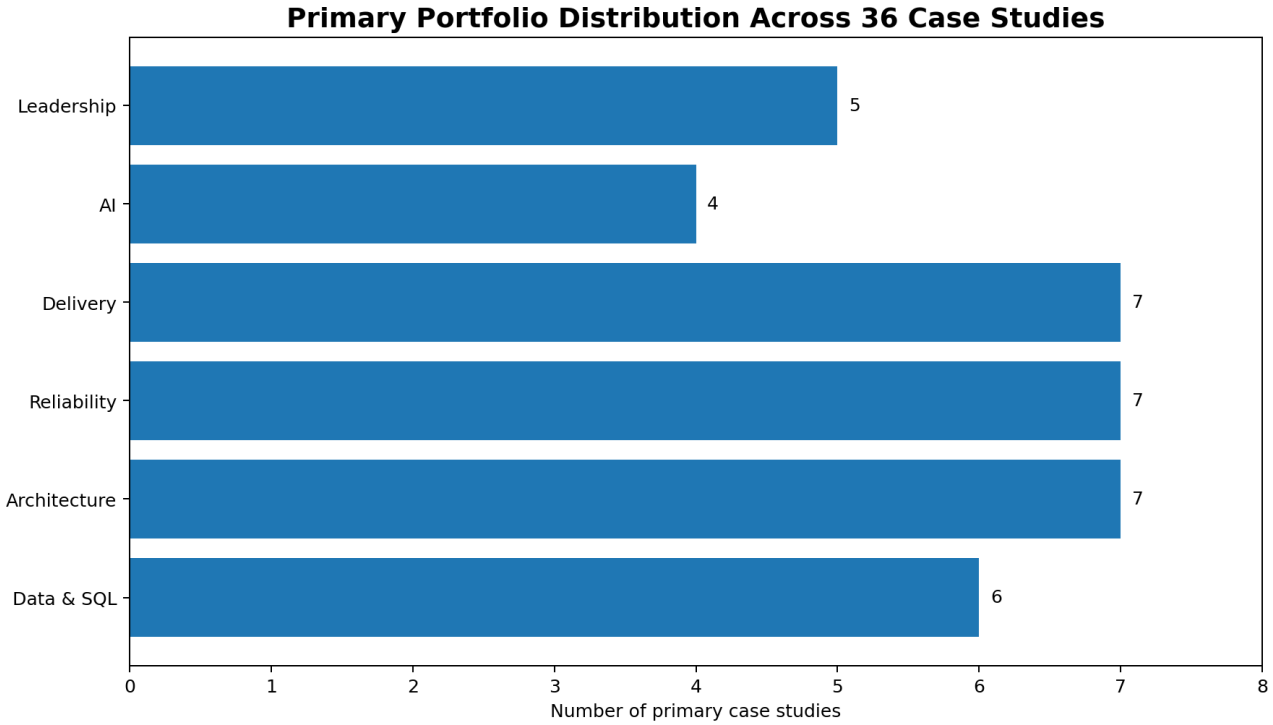


Figure 3. Distribution by primary topic; cross-cutting coverage is broader than the primary count.

4. Complete Case Study Index

#	Case study	Primary group	Core focus
1	Income Limits Processing System	Data and domain	Annual HUD data acquisition, transformation, validation, publication, and seven-year ownership
2	Annual Gross Rent Processing with RabbitMQ	Backend and scale	Asynchronous processing for thousands of tenant records, retries, idempotency, and queue operations
3	Affordable Housing Document Platform and Merge-Field Migration	Data and delivery	High-volume documents, schema redesign, mapping defects, compatibility, and customer impact
4	Production Database Migration and Version 2 Rollout	Delivery and reliability	Database dumps, phased rollout, timestamp evidence, validation, and rollback

#	Case study	Primary group	Core focus
5	Affordable Housing AWS S3 Document Migration	Cloud and delivery	Module-level adaptation of a shared migration framework, QA support, metadata, and storage validation
6	SVN-to-Git Migration and Engineering Enablement	Leadership and modernization	Practical Git adoption, team training, cutover support, and influence without authority
7	Docker Adoption and Microservices Modernization	Modernization	Self-directed container learning, environment consistency, and careful service-boundary reasoning
8	LeaseI - Agentic AI Support Investigation Platform	AI engineering	FastAPI, LangGraph, product knowledge, incidents, live data, read-only diagnosis, and escalation
9	AgentProbe - AI Evaluation and Regression Platform	AI engineering	Benchmarking, tracing, stress testing, regression detection, and AI release confidence
10	SQL Performance Engineering and Production Optimization	Data and performance	Execution plans, indexing, stored procedures, batching, and production-safe tuning
11	Engineering Leadership, Ownership, and Production Operations	Leadership	Long-term ownership, mentoring, reviews, cross-functional delivery, and support
12	Affordable Housing Platform Architecture and Cross-Module Dependencies	Architecture	Module boundaries, shared settings, data flow, downstream consumers, and hidden dependencies
13	Release Engineering, Deployment Validation, and Rollback Strategy	Delivery	Readiness, checkpoints, evidence, go/no-go decisions, and state-aware rollback

#	Case study	Primary group	Core focus
14	Production Incident Investigation, Support Escalation, and Root-Cause Analysis	Reliability and support	Evidence-based debugging, customer scope, hypotheses, containment, and learning
15	Data Engineering, ETL, and Business-Critical Data Workflows	Data engineering	Source ingestion, staging, transformation, quality gates, lineage, and publication
16	Backend API Design, Service Integration, and Asynchronous Workflows	Backend	API contracts, dependency handling, messaging, workers, and resilience
17	Production AI Architecture, Evaluation, and Observability	AI engineering	Retrieval, tools, validation, safety, evaluation, traces, cost, and escalation
18	Testing, Quality Engineering, and Reliability in Business-Critical Systems	Reliability	Unit, integration, database, contract, migration, regression, and production validation
19	System Design, Architecture Trade-offs, and Technical Decision-Making	Architecture	Requirements, quality attributes, boundaries, trade-offs, and evolutionary architecture
20	Security, Compliance, Auditability, and Data Governance in Regulated Systems	Security and governance	Least privilege, tenant isolation, audit evidence, privacy, retention, and regulated boundaries
21	Scalability, Performance, and Capacity Planning for Enterprise Data and AI Systems	Scale and performance	Workload modeling, queue capacity, database scale, AI throughput, and saturation controls
22	Observability, Monitoring, and Operational Excellence	Reliability and operations	Logs, metrics, traces, business signals, alerts, dashboards, runbooks, and SLOs

#	Case study	Primary group	Core focus
23	Configuration Management, Feature Flags, and Cross-Team Dependency Governance	Architecture and operations	Property settings, precedence, drift, change history, feature controls, and hidden dependencies
24	Technical Planning, Estimation, and Cross-Functional Delivery	Leadership and delivery	Requirement discovery, risk estimation, milestones, stakeholder alignment, and production readiness
25	Legacy System Modernization, Refactoring, and Technical Debt Management	Modernization	Characterization, compatibility, phased evolution, retirement, and risk-based prioritization
26	Deterministic Rule Engines, Financial Calculations, and Explainable Business Logic	Data and domain	Eligibility, rent, HAP, effective dating, precision, versioning, and explainability
27	Knowledge Management, Documentation, and Organizational Learning	Leadership and AI	Product knowledge, regulations, incidents, release notes, retrieval, and durable learning
28	Reliability Engineering, Failure Recovery, and Resilient System Design	Reliability	Retries, idempotency, checkpoints, graceful degradation, isolation, and recovery
29	Customer-Centric Engineering, Support Collaboration, and Business Impact	Business impact	Customer incidents, prioritization, safe communication, root cause, and operational value
30	Engineering Enablement, Mentoring, Code Review, and Team Scaling	Leadership	Onboarding, reviews, shared ownership, Git enablement, and reducing expert dependency

#	Case study	Primary group	Core focus
31	Architecture Reviews, Technical Decision Records, and Engineering Governance	Architecture and leadership	Decision framing, ADRs, design review, governance, and production follow-through
32	Multi-Tenant SaaS Architecture, Tenant Isolation, and Shared Platform Design	Architecture and security	Partitioning, authorization, workload fairness, configuration scope, and AI retrieval safety
33	Disaster Recovery, Backup, Restore, and Business Continuity	Reliability	RTO, RPO, database and object restore, queue recovery, exercises, and continuity
34	Data Contracts, Schema Evolution, and Backward Compatibility	Data and delivery	Database, API, message, merge-field, configuration, and AI contract evolution
35	CI/CD Automation, Developer Productivity, and Release Pipelines	Delivery and modernization	Git, build, tests, migrations, Docker, rollout, observability gates, and AI evaluation
36	Cost Optimization, FinOps, and Resource Efficiency	Cost and scale	Unit economics, SQL cost, queue capacity, storage lifecycle, AI cost, and engineering time

5. Data and Domain Engineering Collection

1. Income Limits Processing System

Annual HUD data acquisition, transformation, validation, publication, and seven-year ownership

3. Affordable Housing Document Platform and Merge-Field Migration

High-volume documents, schema redesign, mapping defects, compatibility, and customer impact

10. SQL Performance Engineering and Production Optimization

Execution plans, indexing, stored procedures, batching, and production-safe tuning

15. Data Engineering, ETL, and Business-Critical Data Workflows

Source ingestion, staging, transformation, quality gates, lineage, and publication

26. Deterministic Rule Engines, Financial Calculations, and Explainable Business Logic

Eligibility, rent, HAP, effective dating, precision, versioning, and explainability

34. Data Contracts, Schema Evolution, and Backward Compatibility

Database, API, message, merge-field, configuration, and AI contract evolution

6. Backend, Architecture, and Scale Collection

2. Annual Gross Rent Processing with RabbitMQ

Asynchronous processing for thousands of tenant records, retries, idempotency, and queue operations

12. Affordable Housing Platform Architecture and Cross-Module Dependencies

Module boundaries, shared settings, data flow, downstream consumers, and hidden dependencies

16. Backend API Design, Service Integration, and Asynchronous Workflows

API contracts, dependency handling, messaging, workers, and resilience

19. System Design, Architecture Trade-offs, and Technical Decision-Making

Requirements, quality attributes, boundaries, trade-offs, and evolutionary architecture

21. Scalability, Performance, and Capacity Planning for Enterprise Data and AI Systems

Workload modeling, queue capacity, database scale, AI throughput, and saturation controls

23. Configuration Management, Feature Flags, and Cross-Team Dependency Governance

Property settings, precedence, drift, change history, feature controls, and hidden dependencies

32. Multi-Tenant SaaS Architecture, Tenant Isolation, and Shared Platform Design

Partitioning, authorization, workload fairness, configuration scope, and AI retrieval safety

7. Reliability and Operations Collection

13. Release Engineering, Deployment Validation, and Rollback Strategy

Readiness, checkpoints, evidence, go/no-go decisions, and state-aware rollback

14. Production Incident Investigation, Support Escalation, and Root-Cause Analysis

Evidence-based debugging, customer scope, hypotheses, containment, and learning

18. Testing, Quality Engineering, and Reliability in Business-Critical Systems

Unit, integration, database, contract, migration, regression, and production validation

22. Observability, Monitoring, and Operational Excellence

Logs, metrics, traces, business signals, alerts, dashboards, runbooks, and SLOs

28. Reliability Engineering, Failure Recovery, and Resilient System Design

Retries, idempotency, checkpoints, graceful degradation, isolation, and recovery

33. Disaster Recovery, Backup, Restore, and Business Continuity

RTO, RPO, database and object restore, queue recovery, exercises, and continuity

8. Delivery, Migration, and Modernization Collection

4. Production Database Migration and Version 2 Rollout

Database dumps, phased rollout, timestamp evidence, validation, and rollback

5. Affordable Housing AWS S3 Document Migration

Module-level adaptation of a shared migration framework, QA support, metadata, and storage validation

6. SVN-to-Git Migration and Engineering Enablement

Practical Git adoption, team training, cutover support, and influence without authority

7. Docker Adoption and Microservices Modernization

Self-directed container learning, environment consistency, and careful service-boundary reasoning

24. Technical Planning, Estimation, and Cross-Functional Delivery

Requirement discovery, risk estimation, milestones, stakeholder alignment, and production readiness

25. Legacy System Modernization, Refactoring, and Technical Debt Management

Characterization, compatibility, phased evolution, retirement, and risk-based prioritization

35. CI/CD Automation, Developer Productivity, and Release Pipelines

Git, build, tests, migrations, Docker, rollout, observability gates, and AI evaluation

36. Cost Optimization, FinOps, and Resource Efficiency

Unit economics, SQL cost, queue capacity, storage lifecycle, AI cost, and engineering time

9. AI Engineering Collection

8. LeaseI - Agentic AI Support Investigation Platform

FastAPI, LangGraph, product knowledge, incidents, live data, read-only diagnosis, and escalation

9. AgentProbe - AI Evaluation and Regression Platform

Benchmarking, tracing, stress testing, regression detection, and AI release confidence

17. Production AI Architecture, Evaluation, and Observability

Retrieval, tools, validation, safety, evaluation, traces, cost, and escalation

27. Knowledge Management, Documentation, and Organizational Learning

Product knowledge, regulations, incidents, release notes, retrieval, and durable learning

10. Leadership and Business Impact Collection

11. Engineering Leadership, Ownership, and Production Operations

Long-term ownership, mentoring, reviews, cross-functional delivery, and support

24. Technical Planning, Estimation, and Cross-Functional Delivery

Requirement discovery, risk estimation, milestones, stakeholder alignment, and production readiness

27. Knowledge Management, Documentation, and Organizational Learning

Product knowledge, regulations, incidents, release notes, retrieval, and durable learning

29. Customer-Centric Engineering, Support Collaboration, and Business Impact

Customer incidents, prioritization, safe communication, root cause, and operational value

30. Engineering Enablement, Mentoring, Code Review, and Team Scaling

Onboarding, reviews, shared ownership, Git enablement, and reducing expert dependency

31. Architecture Reviews, Technical Decision Records, and Engineering Governance

Decision framing, ADRs, design review, governance, and production follow-through

11. Foundational Production Examples

Several real situations appear repeatedly because they demonstrate different senior-level competencies depending on the lens used.

Production example	Technical facts	Portfolio themes supported
Annual HUD income limits	A few thousand source records transformed into approximately 180,000 records; seven years of ownership	ETL, SQL, domain rules, data quality, performance, releases, reliability, leadership
24-hour eligibility investigation	Affordable Housing logic and data were correct; another team's property-level setting caused ineligibility	Incident response, configuration governance, cross-team dependencies, LeaseI motivation, customer impact
Document merge-field redesign	Entire database design changed; hundreds of documents generated daily; six urgent defects in one day	Schema evolution, backward compatibility, testing, customer support, migration governance

Production example	Technical facts	Portfolio themes supported
Version 2 production migration	Initial concern that migration was wrong; database dumps and timestamps proved the actual state	Evidence-based debugging, rollback planning, release engineering, architecture governance
Annual gross-rent processing	Thousands of tenant records processed asynchronously with RabbitMQ concepts	Messaging, idempotency, retries, capacity, observability, cost
AWS S3 document migration	Owned the Affordable Housing module adaptation, integration, QA support, and validation	Cloud integration, boundaries, documents, delivery, tenant isolation
SVN-to-Git transition	Learned Git, delivered KT sessions, supported team adoption and migration	Influence, mentoring, developer productivity, modernization
Docker self-learning	Learned Dockerfiles, Compose, images, containers, volumes, and ports through personal projects	Initiative, environment consistency, modernization, CI/CD

12. Experience-to-Competency Matrix

Competency	Primary supporting case studies
Complex SQL and data engineering	1, 10, 15, 26, 34, 36
Backend and asynchronous systems	2, 16, 21, 28, 35
System design and architecture	12, 19, 23, 31, 32
Production reliability	13, 14, 18, 22, 28, 33
Migrations and modernization	3, 4, 5, 7, 25, 34, 35
Security and regulated systems	20, 26, 32, 33
AI engineering and evaluation	8, 9, 17, 27, 35, 36
Leadership and influence	6, 11, 24, 29, 30, 31
Customer and business impact	1, 3, 14, 26, 29
Operational and cost judgment	21, 22, 28, 33, 36

13. Suggested Navigation Paths

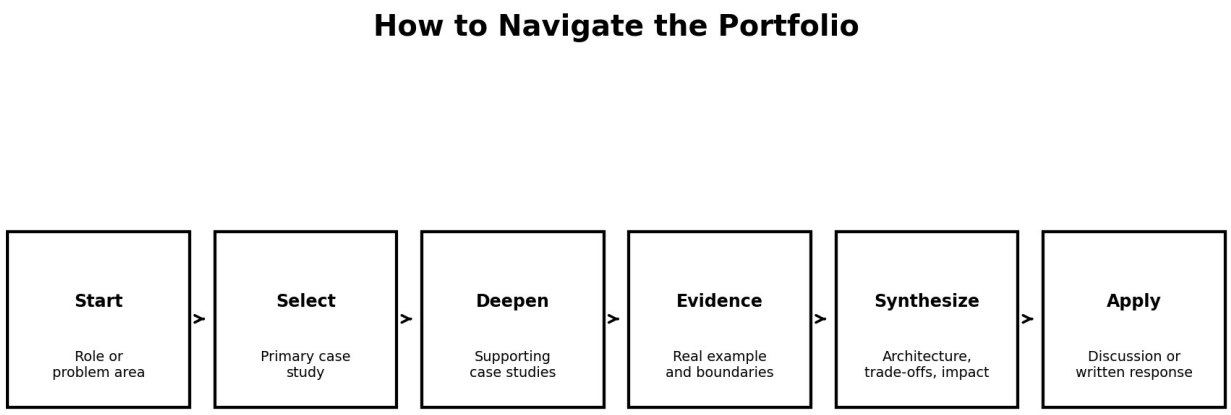


Figure 4. Start from the role or problem, then combine one primary and two supporting case studies.

Discussion focus	Start with	Then deepen with
Senior data engineering	1 or 15	10, 26, 34, 36
Backend and distributed systems	16	2, 21, 28, 35
Architecture and design judgment	19	12, 23, 31, 32
Production ownership	11	13, 14, 22, 28
Database migration	4	13, 18, 25, 34
AI platform architecture	8 or 17	9, 20, 27, 35, 36
Leadership and influence	30	6, 11, 24, 31
Customer impact	29	3, 14, 23, 26
Reliability and operations	28	14, 18, 22, 33
Modernization strategy	25	5, 6, 7, 34, 35

14. Evidence and Claim Boundaries

- Direct experience is stated as built, owned, supported, learned, reviewed, or delivered.
- Shared platform work is separated from module-level ownership. The AWS S3 example explicitly states ownership of the Affordable Housing portion, not the shared migration framework.
- RabbitMQ was not originally the author's idea; the portfolio emphasizes rapid learning, contribution, and deep understanding rather than invention.

- Docker experience is hands-on self-learning and personal-project implementation, followed by informed recommendation. It is not presented as ownership of an enterprise container platform.
- LeaseI is separated into current implementation and target architecture. Current retrieval, storage, confidence, and concurrency limitations remain visible.
- AI confidence values are not described as statistically calibrated when they are hand-tuned constants.
- Regulated decisions remain deterministic and human-governed. AI is positioned for understanding, retrieval, summarization, diagnosis support, and escalation.

These boundaries improve credibility. Senior-level strength comes from clear ownership and judgment, not from claiming every adjacent platform responsibility.

15. Cross-Study Themes

Correctness before convenience

Regulated and financial workflows require deterministic validation, historical reproducibility, and explicit failure handling.

Evidence before assumption

Database dumps, timestamps, counts, traces, configuration history, and audit records guide production decisions.

State-aware engineering

Code, data, schema, messages, documents, configuration, and AI artifacts form one production state.

Incremental modernization

Compatibility windows, phased rollouts, feature controls, checkpoints, and retirement reduce risk.

Business observability

Infrastructure health is insufficient when published data, calculations, documents, or AI conclusions are incorrect.

Human-centered AI

AI assists investigation and synthesis while deterministic systems and people retain high-impact authority.

Leadership through enablement

Knowledge transfer, reviews, planning, documentation, and shared ownership scale engineering capacity.

16. Portfolio Strengths

Strength	Why it is differentiated
Deep domain ownership	The work connects technical decisions to eligibility, rent, HAP, tax credits, documents, and customer outcomes.
Production credibility	The collection includes incidents, migrations, urgent defects, rollback, support, and evidence - not only greenfield designs.

Strength	Why it is differentiated
Data-to-platform range	SQL and ETL depth extend into APIs, queues, cloud storage, multitenancy, reliability, and CI/CD.
Modern AI judgment	LeaseI and AgentProbe are framed around retrieval, tools, evaluation, observability, boundaries, and human escalation.
Honest ownership boundaries	Shared frameworks, original ideas, self-learning, and target architecture are clearly distinguished.
Leadership without title dependence	The examples show mentoring, Git adoption, reviews, planning, and influence through technical credibility.

17. Remaining Optional Extensions

The portfolio is comprehensive. Additional documents would be narrower syntheses rather than major new competency areas.

- API versioning and developer-experience handbook.
- Data lineage and metadata-management operating model.
- Vendor evaluation and third-party integration governance.
- Engineering metrics and organizational effectiveness scorecard.
- Executive one-page summaries for selected flagship case studies.
- A combined portfolio package with a cover, index, and merged appendices.

These are optional because their primary technical content already appears across the 36 detailed case studies.

18. Portfolio Maintenance Guide

- Update implementation status when LeaseI or AgentProbe changes materially.
- Preserve the distinction between production experience and proposed modernization.
- Add measurable outcomes only when evidence is available.
- Keep names, dates, technology versions, and scale statements consistent across documents.
- Retire duplicate explanations rather than expanding the collection indefinitely.
- Use this master index as the canonical navigation source for future revisions.

19. Closing Synthesis

The complete collection shows a progression from implementation to ownership, from ownership to architecture, and from architecture to organizational impact. The technical foundation is data-intensive Affordable Housing production work. The senior-level differentiation is the ability to connect that foundation to reliability, delivery, modernization, AI, customer outcomes, and team capability.

Taken together, the case studies support a coherent profile: an engineer who understands why a system exists, how its data and dependencies behave, how to release and operate it safely, how to investigate it with evidence, how to modernize it without abandoning customers, and how to help other engineers become capable owners.